

Manual Tecnico TicketLite

ADSO

Valeria Andrea Osorio Santana

Juliana Rodriguez Castrillon

SENA - Itagüí

2901483

2025

Introducción

Descripción General del Sistema

TICKETLITE es una aplicación web que permite la gestión y administración de eventos, orientada tanto a usuarios que desean asistir a actividades como a administradores que las crean y organizan. El sistema está diseñado bajo una arquitectura cliente-servidor, lo que permite la interacción entre la interfaz gráfica y la capa lógica a través de peticiones HTTP.

Arquitectura del Sistema

El sistema está basado en el patrón Modelo-Vista-Controlador (MVC). La vista se encuentra implementada con el framework Vue.js para ofrecer una interfaz dinámica y amigable. La capa lógica del negocio está desarrollada en Java utilizando Spring Boot, que gestiona las peticiones del cliente y la comunicación con la base de datos. El modelo corresponde a la estructura de datos persistida en PostgreSQL, accedida mediante herramientas especializadas para administración y consultas.

Herramientas y Tecnologías Utilizadas

El desarrollo del sistema se realizó utilizando los lenguajes JavaScript para el frontend y Java para el backend. En la capa de servidor se empleó Spring Boot como framework principal, mientras que en el cliente se implementó Vue.js para la construcción de la interfaz. Para el diseño de las pantallas se utilizó Figma, permitiendo prototipado y diseño colaborativo. La administración del control de versiones se realizó mediante repositorios en GitHub y GitLab, asegurando la correcta gestión del código y la colaboración entre desarrolladores. La base de datos se gestionó en PostgreSQL y se utilizó la herramienta SQL Maestro para la administración y optimización de consultas.

Instalación del Sistema

Para la instalación del sistema, se debe contar con Java Development Kit (JDK) en su versión 17 o superior, Node.js para la ejecución del frontend y PostgreSQL para la base de datos. Luego de clonar el repositorio desde GitHub o GitLab, es necesario instalar las dependencias del proyecto Vue.js mediante el comando `npm install` y compilarlo para producción. En el backend, el proyecto Spring Boot debe configurarse con las credenciales de conexión a la base de datos en el archivo `application.properties`. Posteriormente, se deben generar las tablas mediante los scripts SQL definidos y, finalmente, ejecutar la aplicación desde el IDE o mediante la línea de comandos con el comando `mvn spring-boot:run`.

Configuración del Sistema

La configuración incluye la definición de variables para la conexión a la base de datos, la configuración del puerto del servidor y la habilitación de CORS para

permitir la interacción entre el frontend y el backend. También es necesario configurar los repositorios remotos en Git para garantizar la correcta sincronización de los cambios entre entornos de desarrollo y producción.

Base de Datos

El sistema utiliza PostgreSQL como motor principal, administrado mediante la herramienta SQL Maestro, que facilita la ejecución de consultas y la gestión de esquemas. Las tablas principales incluyen usuarios, eventos y notificaciones, definidas mediante claves primarias y relaciones entre entidades para garantizar la integridad de los datos.

Despliegue del Sistema

Para el despliegue en producción, el backend desarrollado en Spring Boot se ejecuta en un servidor Java, mientras que el frontend construido con Vue.js se compila y se sirve desde un servidor web. Se recomienda utilizar un proxy inverso como Nginx para la distribución del contenido estático y la gestión de peticiones. La base de datos debe estar alojada en un entorno seguro con acceso restringido y respaldos automáticos.

Mantenimiento del Sistema

El mantenimiento se basa en la actualización de dependencias, la revisión periódica de logs del sistema y la ejecución de pruebas unitarias y de integración. Es importante garantizar la integridad de la base de datos mediante copias de seguridad y optimizar consultas SQL para evitar sobrecargas en el servidor.

Solución de Problemas Comunes

Si el backend no se inicia, se debe verificar que la conexión a la base de datos esté activa y que las credenciales en `application.properties` sean correctas. Si el frontend no carga, se recomienda revisar la instalación de dependencias en Node.js y reconstruir el proyecto con `npm run build`. En caso de errores de CORS, se debe revisar la configuración en el proyecto Spring Boot y habilitar los dominios permitidos.

Documentación del Código

El código se encuentra estructurado en carpetas que separan el modelo, las vistas y los controladores, siguiendo la arquitectura MVC. Los controladores gestionan las peticiones HTTP, el modelo representa las entidades de la base de datos y las vistas están implementadas en Vue.js. El proyecto cuenta con repositorios en GitHub y GitLab para su control de versiones y un archivo README que describe la estructura y comandos principales para la ejecución.

Conclusión

TICKETLITE representa una solución integral para la gestión de eventos, combinando una arquitectura robusta basada en el patrón MVC con tecnologías modernas que garantizan escalabilidad, rendimiento y facilidad de uso. Su diseño modular, sustentado en herramientas como Spring Boot, Vue.js y PostgreSQL, permite una interacción fluida entre las diferentes capas del sistema y asegura la integridad de los datos.

El uso de repositorios Git asegura un desarrollo colaborativo eficiente y controlado, mientras que las prácticas de despliegue y mantenimiento establecidas proporcionan estabilidad y seguridad en entornos de producción. Gracias a su enfoque bien estructurado y a la correcta configuración de cada componente, TICKETLITE se consolida como una aplicación confiable y adaptable a las necesidades de los usuarios y administradores, ofreciendo una experiencia completa para la organización y asistencia a eventos.

Requerimientos Funcionales:

- 1- Permitir a los usuarios registrados recuperar su contraseña por correo electrónico.
- 2- Permitir a los usuarios registrarse
- 3- Permitir al usuario salir de su cuenta de forma segura.
- 4- Capturar información de los asistentes durante el proceso de registro.
- 5- Generar y enviar confirmaciones de registro por correo electrónico.
- 6- Permitir a los usuarios editar su perfil de registro.
- 7- Gestionar Login ok.
- 8- Mostrar mensaje de error cuando los datos ingresados del usuario sean incorrectos.
- 9- Mostrar un calendario con todos los eventos disponibles
- 10- Permitir a los usuarios filtrar eventos por fecha.
- 11- Permitir a los usuarios guardar eventos en su lista de eventos favoritos
- 12- Integrar recordatorios automáticos para eventos guardados
- 13- Mostrar información detallada sobre los eventos y las opciones de entrada
- 14- Permitir al usuario registrar su asistencia a un evento
- 15- Ofrecer un campo de localización para ingresar una ciudad, estado o país y devolver los eventos que coincidan.
- 16- Mostrar eventos cercanos a la ubicación actual del usuario
- 17- Permitir que cada evento muestre claramente su ciudad y dirección
- 18- Permitir que cada evento muestre su ubicación a través de un mapa
- 19- Permitir mostrar una imagen del estadio o lugar donde se realizará el evento
- 20- Permitir a los usuarios seleccionar y comprar entradas para eventos.
- 21- Ofrecer diferentes métodos de pago seguro

- 22- Generar y enviar entradas electrónicas a través de un código QR por correo electrónico
- 23- Permitir la cancelación y reembolso de entradas según la política del evento.
- 24- Mostrar mensaje de confirmación tras completar la compra de entradas
- 25- Permitir enviar por correo electrónico una confirmación de cancelación de ticket
- 26- Permitir mostrar ticket virtual con el código qr respectivo de la compra
- 27- Integrar funciones de redes sociales para compartir eventos
- 28- Permitir a los usuarios comentar y calificar eventos
- 29- Permitir a los organizadores agregar información sobre oradores y presentadores.
- 30- Programar y organizar sesiones, charlas y actividades dentro de los eventos
- 31- Notificar a los oradores sobre detalles y cambios en su participación.
- 32- Mostrar horarios y detalles de las sesiones en la aplicación.
- 33- Agregar información de los eventos antes de ser publicados
- 34- Gestionar las imágenes del sistema en un storage en línea.
- 35- Permitir a los organizadores ver el número de asistentes registrados en un evento
- 36- Permitir al administrador crear un nuevo evento
- 37- Permitir al administrador actualizar la información de un evento
- 38- Permitir al administrador eliminar un evento
- 39- Ofrecer recomendaciones de eventos basadas en intereses y comportamiento pasado.
- 40- Permitir administrar acuerdos de patrocinio para eventos mediante una app externa.
- 41- Mostrar información sobre oradores y sesiones en la aplicación.
- 42- Reporte de estadísticas para administradores
- 43- Enviar recordatorios automáticos sobre eventos próximos.
- 44- Notificar a los usuarios sobre cambios en eventos o horarios.
- 45- Permitir a los organizadores enviar mensajes a los asistentes mediante una notificación.
- 46- Facilitar la comunicación bidireccional entre organizadores y asistentes mediante una app externa.
- 47- Permitir enviar recordatorio 24 horas antes del evento
- 48- Recopilar datos sobre la asistencia y participación en eventos.
- 49- Analizar datos para identificar tendencias y patrones de comportamiento.
- 50- Generar informes de estadísticas sobre el rendimiento de los eventos y la satisfacción del usuario.

JIRA: <https://09876543hgy.atlassian.net/jira/core/projects/BAC/timeline>

Plantillas Extendidas de Casos de Uso:

<https://docs.google.com/document/d/1IUsFQnuj3j9KrUlxNb7OKyEcpw47O9FG/edit#heading=h.l4pxql0ehvwu>

Instalacion y Configuracion:

<https://youtu.be/C4MenVXMIBY>

EndPoints | API:

<https://youtu.be/jq9FoB85IHA>

Funcionamiento de Todo el Frontend:

<https://youtu.be/MiVSbOp39JE>